Dostupné škálovateľné riešenia pre spracovanie veľkého objemu dát a dátové sklady

Vyhľadávanie informácií

Martin Šeleng

Horizontálne škálovanie

- Google
  - Ako prvý postavil škálovateľný systém z bežných PC
- Moorove pravidlo
  - http://en.wikipedia.org/wiki/Moore's_law
  - obsah internetu rastie rýchlo ale pomalšie ako veľkosť diskovej kapacity a výkonnosť obyčajných PC
  - výkonné počítače sú drahé a rýchlo zastarajú
- Typy distribuovaných architektúr
  - Share memory
  - Share CPU/GPU
  - Share disk
  - Share nothing

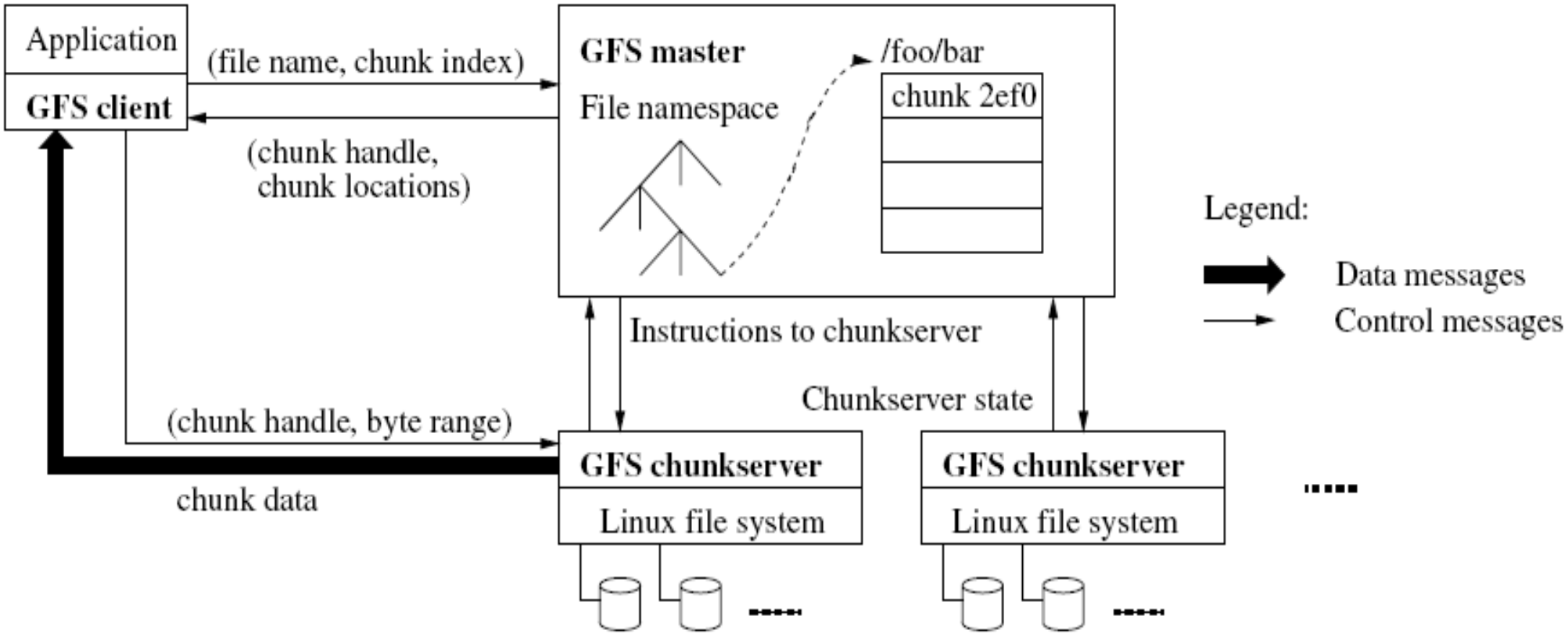
Distribuované súborové systémy

Obsah

- Google File System (GFS)
- Hadoop File System (HDFS)
- Simple Storage Service (S3)

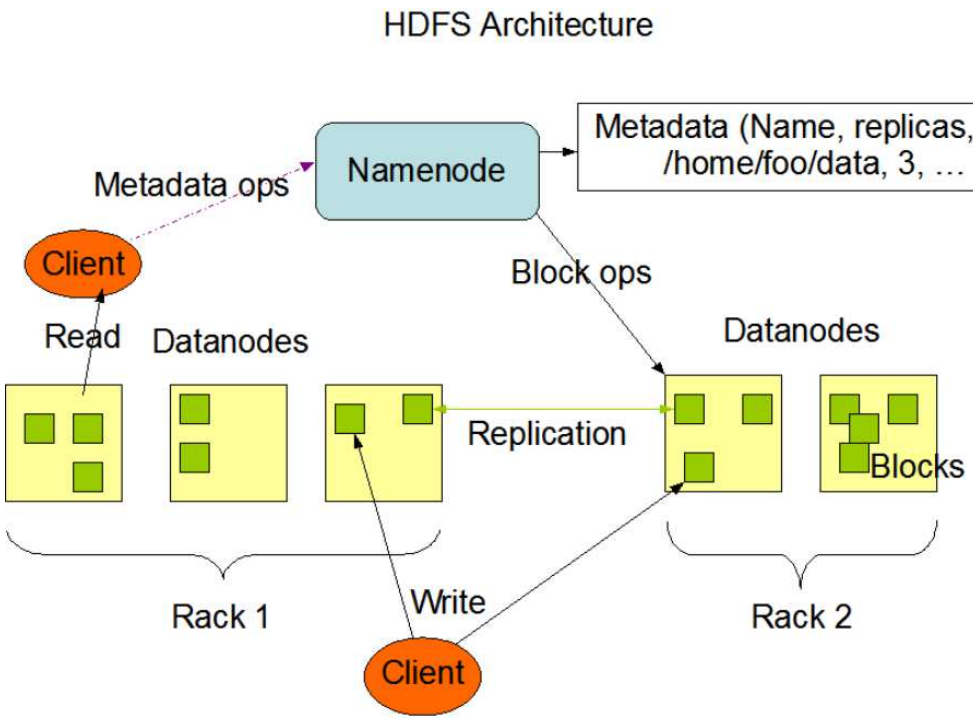
Distribuovaný súborový systém GFS

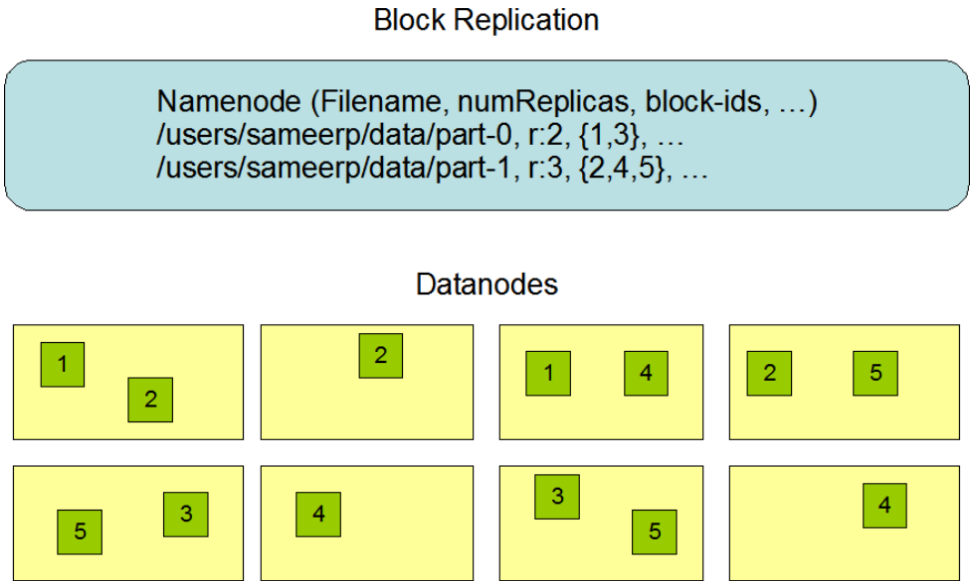
- Master/Slave architektúra GFS (Google File System)



Distribuovaný súborový systém HDFS

- Master/Slave architektúra HDFS (Hadoop Distributed File System)
- Replikácia blokov dát





Príkazy v prostredí HDFS

- Operácie HDFS - /bin/hadoop fs
  - [-ls <path>]
  - [-lsr <path>]
  - [-du <path>]
  - [-dus <path>]
  - [-mv <src> <dst>]
  - [-cp <src> <dst>]
  - [-rm <path>]
  - [-rmr <path>]
  - [-expunge]
  - [-put <localsrc> <dst>]
  - [-copyFromLocal <localsrc> <dst>]
  - [-moveFromLocal <localsrc> <dst>]
  - [-get [-crc] <src> <localdst>]
  - [-getmerge <src> <localdst> [addnl]]
  - [-cat <src>]
  - [-text <src>]
  - [-copyToLocal [-crc] <src> <localdst>]
  - [-moveToLocal [-crc] <src> <localdst>]
  - [-mkdir <path>]
  - [-setrep [-R] [-w] <rep> <path/file>]
  - [-touchz <path>]
  - [-test -[ezd] <path>]
  - [-stat [format] <path>]
  - [-tail [-f] <file>]
  - [-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH...]
  - [-chown [-R] [OWNER][:[GROUP]] PATH...]
  - [-chgrp [-R] GROUP PATH...]
  - [-help [cmd]]

Distribuovaný súborový systém S3

- Tento systém je založený výlučne na REST službách (HTTP) a protokole SOAP
- Dáta sú uložené v regiónoch najbližších k používateľovi
- Bez súhlasu používateľa nie sú dáta migrované do iných regiónov (ani kvôli zabezpečeniu replík)
- Založený na existujúcich súborových systémoch (FAT, NTFS, EXT3, EXT4, atď.)

MAP/REDUCE

Obsah

- Úvod
- MapReduce framework (Google a Hadoop)
- Ukážka programu spracovávaného v MapReduce klasteri
- Spustenie programu na MapReduce klastri

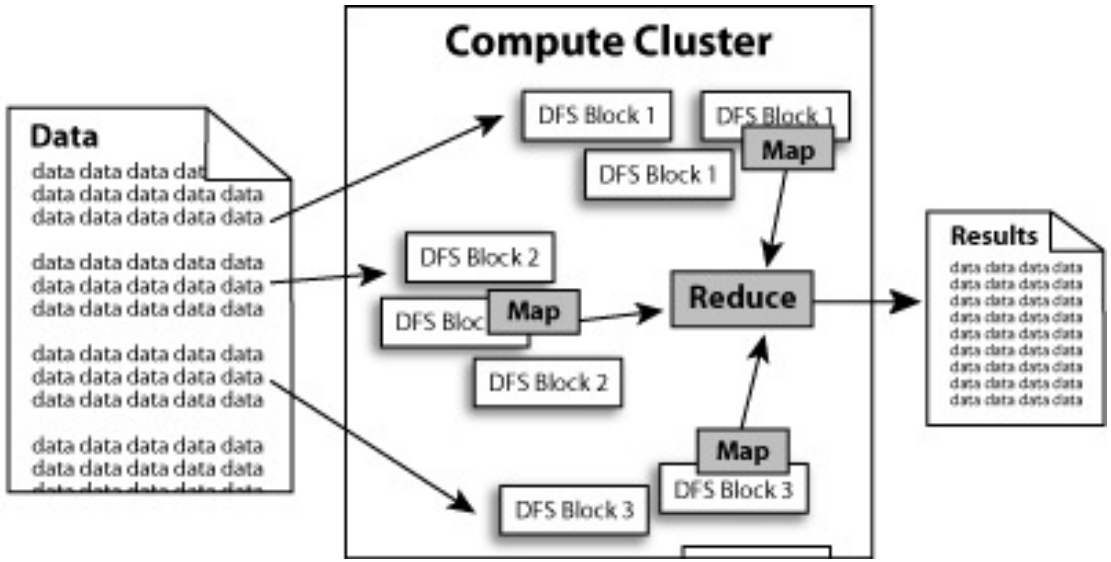
Úvod

- Architektúra
  - spôsob distribuovania dát,
  - množstvo a spôsob replikácie dát,
  - spôsob (framework) paralelného spracovania,
  - množstvo a typ uzlov, na ktorých bude spracovanie prebiehať,
  - jednoducho škálovateľný systém, ktorý by bol efektívny a spoľahlivý.

- Existujúce systémy na spracovanie veľkého množstva informácií
  - Parallel Virtual Machine (PVM)
  - Message Passing Interface (MPI)
  - Condor - High Throughput Computing (HTC)
  - Gridové riešenia
  - Cloudové riešenia

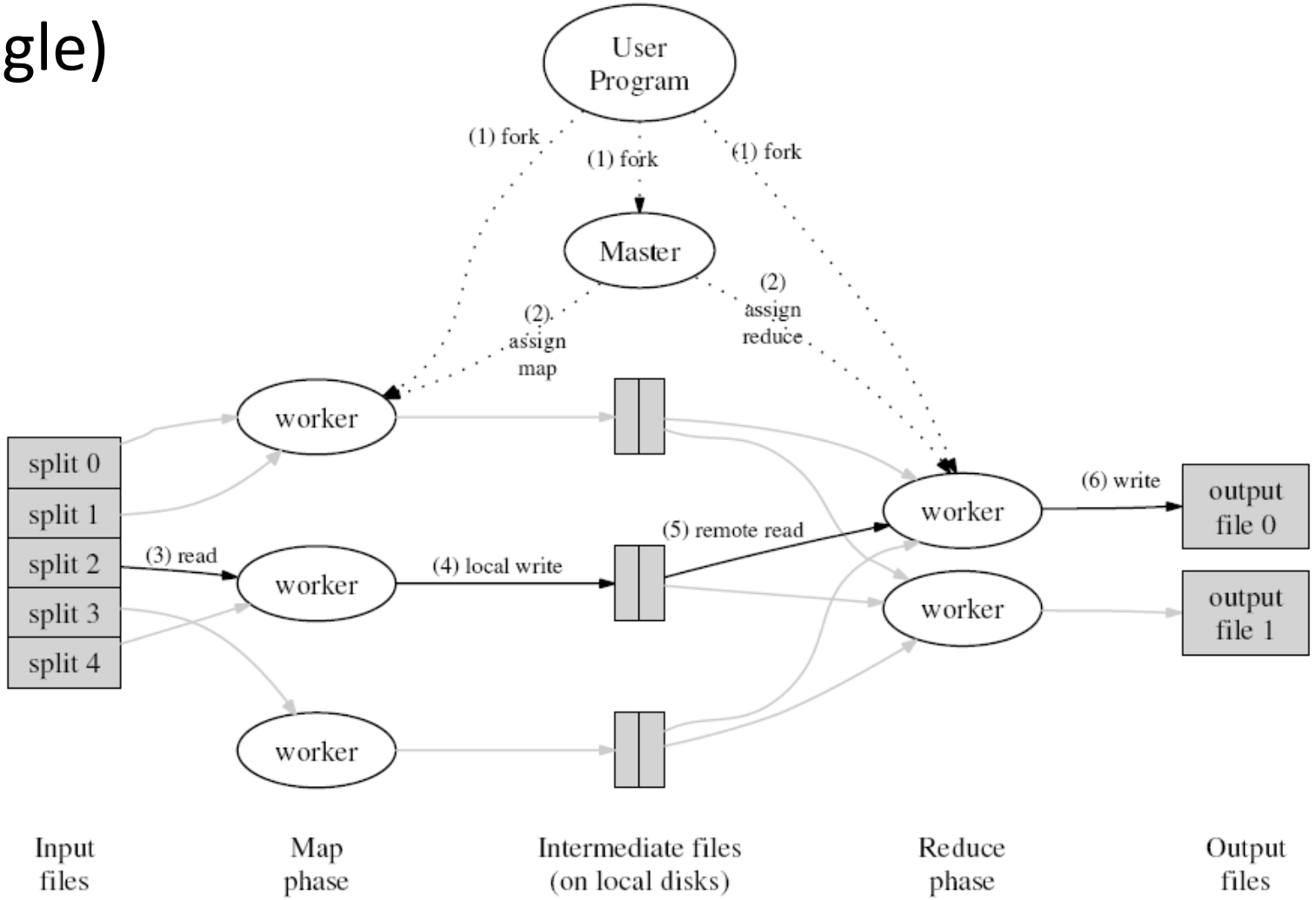
Architektúra

- Google idea
  - dvojjadrové x86 procesory, beží na nich operačný systém Linux a majú 2-4GB pamäte
  - sieťové pripojenie s rýchlosťou 100Mb/s
  - Kluster pozostáva zo stoviek až tisícok pracovných staníc
  - disky s IDE rozhraním
  - Používateľ posiela do systému procesy, ktoré plánovač rozdelí na voľné pracovné stanice a spustí



MapReduce framework (Google a Hadoop) (1)

- Spustenie a vykonanie procesu v prostredí MapReduce (Google)



MapReduce framework (Google a Hadoop) (2)

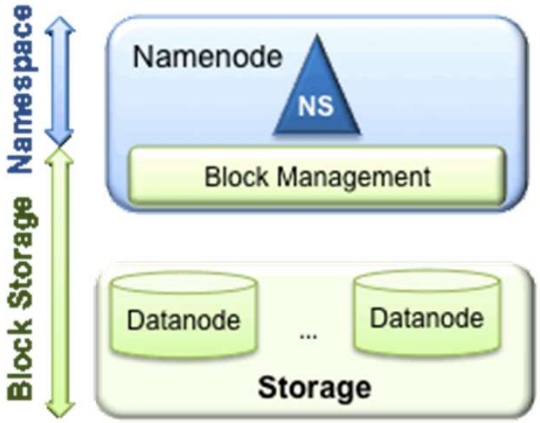
- Spustenie a vykonanie procesu v prostredí MapReduce (Hadoop)
  - **Na uzle JobTracker sa spustí požadovaný proces, ktorý má naimplementované funkcie Map a Reduce.**
  - **JobTracker preskúma voľné uzly a podľa potreby (v závislosti od veľkosti vstupných dát) pridelí potrebné množstvo výpočtových uzlov (TaskTracker v závislosti od počtu jadier zvládne počítať 2 až 4 úlohy naraz). Súčasne je spustená aj úloha Reduce (v závislosti od množstva dát a uzlov sa môže spustiť aj viac úloh Reduce).**
  - **Po dokončení niektorej z Map úloh sa jej výsledky prekopírujú na niektorý z uzlov, kde beží úloha Reduce. Výsledky sa utriedia a čaká sa na ukončenie všetkých úloh Map.**
  - **Po dokončení všetkých úloh Map sa spustia úlohy Reduce a po ich ukončení dostaneme utriedený zoznam párov kľúč/hodnota.**

HDFS – nová architektúra Apache Hadoop 2.0.x (1)

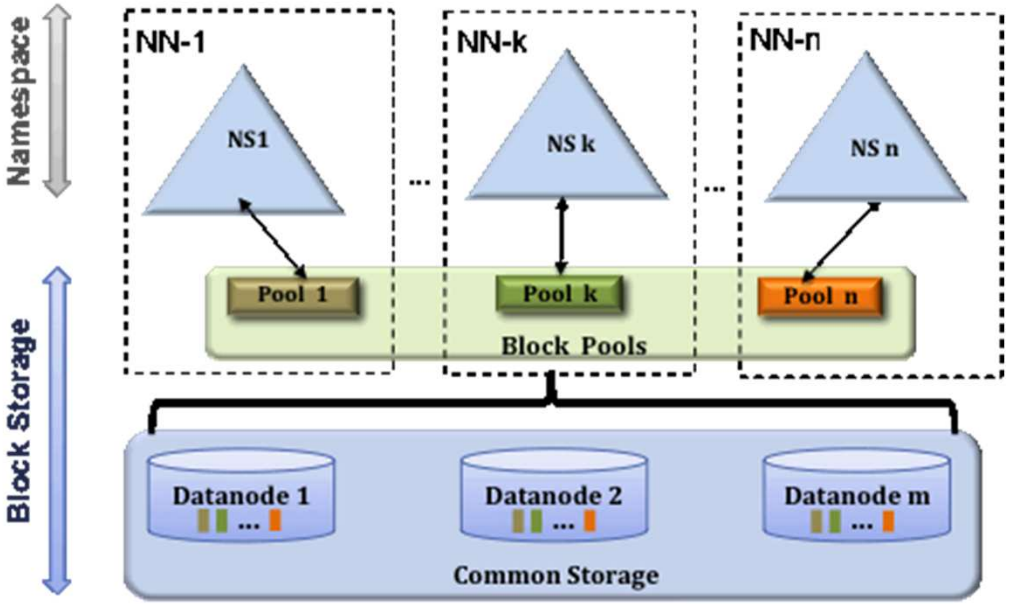
- V predchádzajúcej architektúre HDFS sme mali dosť veľký problém so serverom Namenode, ktorý tvoril tzv. SPOF (Single Point Of Failure)
- Ak nám tento server spadol, celý distribuovaný systém bol nefunkčný, a tým pádom aj celý MapReduce framework stratil zmysel
- V prípade predstavenej architektúry síce existovalo niečo ako Secondary Namenode, avšak tento slúžil, len na zálohovanie metadát, teda v prípade výpadku primárneho Namenode nepreberal automaticky jeho funkciu
- Toto sa ukázalo ako veľký problém, a preto bola predstavená nová architektúra, kde bola vytvorená tzv. HDFS Federation (federácia Namenode serverov)

HDFS – nová architektúra Apache Hadoop 2.0.x (2)

- HDFS obsahuje 2 základné vrstvy
  - Namespace
    - Pozostávajúca s adresárov, súborov a blokov
    - Podporujúca základné operácie ako: vytvorenie, zmazanie, modifikácia a poskytnutie zoznamu súborov a adresárov
  - Block Storage Service obsahuje 2 časti
    - Block management (poskytovaný serverom Namenode)
      - Poskytovanie registrácie serverov Datanode
      - Udržiavanie informácií o blokoch
      - Poskytovanie operácii nad blokmi: vytvorenie, zmazanie, modifikácia a získanie lokality bloku
      - Manažovanie replík a ich umiestnenia
    - Storage, ktorý je poskytovaný servermi Datanode podporujúcimi operácie read/write

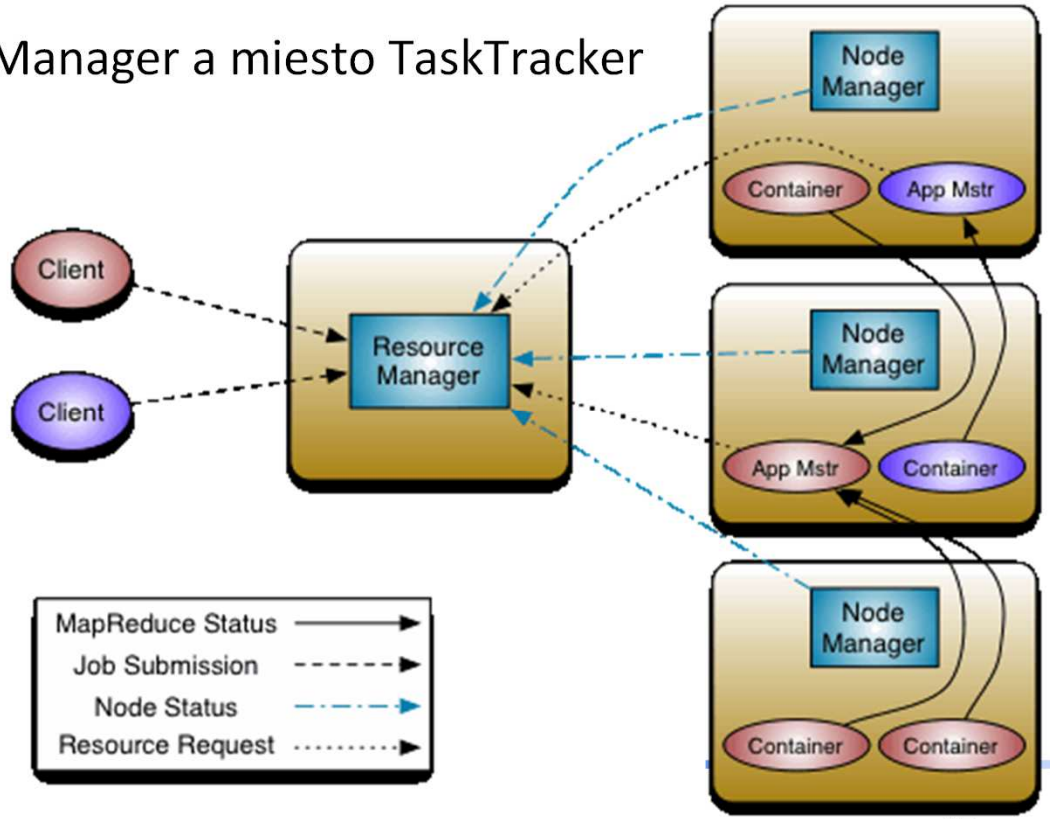


- Viacero Namenode/Namespace serverov
  - Na horizontálne škálovanie federácia požíva viacero nezávislých Namenodes/ Namespaces serverov
  - Namenode servery o sebe navzájom nevedia
  - Každý Datanode server sa registruje u všetkých Namenode serverov
  - Výpadok jedného neovplyvní činnosť distribuovaného systému
  - V prípade federácie preťaženie jedného Namenode servera nespôsobí výpadok celého distribuovaného úložného systému



Apache Hadoop NextGen MapReduce (YARN)

- V prípade predchádzajúcej generácie MapReduce (MRv1), sme mali JobTracker server a TaskTracker servery (bežiace na uzloch tvoriacich HDFS)
- V novej verzii (YARN MRv2) sa dve hlavné funkcionality JobTracker servera manažovanie prostriedkov a plánovanie/monitorovanie spustených úloh rozdelili na 2 časti:
  - **Globálny ResourceManager (RM)**
  - **Pre každú aplikáciu ApplicationMaster (AM) – aplikácia je úloha (ako v MRv1) alebo DAG (Direct Acyclic Graph) úloh**
- Miesto JobTracker servra máme ResourceManager a miesto TaskTracker serverov NodeManager (NM) servery
- RM je rozhoduje o prideľovaní zdrojov pre všetky aplikácie bežiace v systéme
- Každý AM je zodpovedný za vyjednávanie nových zdrojov od RM a pracuje spolu s NM pri vykonávaní a monitorovaní jednotlivých taskov.
- RM má 2 základné komponenty:
  - **Plánovač**
  - **Manažér aplikácií**



Ukážka programu spracovávaného v MapReduce klusteri (1)

- Funkcia Map:

```
public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable> output,
    Reporter reporter) throws IOException {
  Text word = new Text();
  String line = value.toString().toLowerCase();
  StringTokenizer tokenizer = new StringTokenizer(line);
  while (tokenizer.hasMoreTokens()) {
    word.set(tokenizer.nextToken());
    output.collect(word, one);
  }
}
```

- Funkcia Reduce:

```
public void reduce(Text key, Iterator<IntWritable> values, OutputCollector<Text, IntWritable>
    output, Reporter reporter) throws IOException {
  int sum = 0;
  while (values.hasNext())
    sum += values.next().get();
  output.collect(key, new IntWritable(sum));
}
```

Ukážka programu spracovávaného v MapReduce klusteri (2) – Mapper

```
#!/usr/bin/python
"""mapper.py"""
import sys
# input comes from STDIN (standard input)
for line in sys.stdin:
    # remove leading and trailing whitespace
    line = line.strip()
    # split the line into words
    words = line.split()
    # increase counters
    for word in words:
        # write the results to STDOUT (standard output);
        # what we output here will be the input for the
        # Reduce step, i.e. the input for reducer.py
        #
        # tab-delimited; the trivial word count is 1
        print('%s\t%s' % (word, 1))
```

Ukážka programu spracovávaného v MapReduce klusteri (3) – Reducer

```
#!/usr/bin/env python
"""reducer.py"""
from operator import itemgetter
import sys
current_word = None
current_count = 0
word = None
# input comes from STDIN
for line in sys.stdin:
    # remove leading and trailing whitespace
    line = line.strip()
    # parse the input we got from mapper.py
    word, count = line.split('\t', 1)
    # convert count (currently a string) to int
    try:
        count = int(count)
except ValueError:
        # count was not a number, so silently
        # ignore/discard this line
        continue
    # this IF-switch only works because Hadoop
    sorts map output
    # by key (here: word) before it is passed to the
    reducer
    if current_word == word:
        current_count += count
    else:
        if current_word:
            # write result to STDOUT
            print('%s\t%s' % (current_word,
    current_count))
        current_count = count
        current_word = word
# do not forget to output the last word if
    needed!
if current_word == word:
    print('%s\t%s' % (current_word,
    current_count))
```

Ukážka programu spracovávaného v MapReduce klusteri (4) – Mapper

```
#!/usr/bin/env python
"""A more advanced Mapper, using Python iterators and generators."""
import sys
def read_input(file):
    for line in file:
        # split the line into words
        yield line.split()
def main(separator='\t'):
    # input comes from STDIN (standard input)
    data = read_input(sys.stdin)
    for words in data:
        # write the results to STDOUT (standard output);
        # what we output here will be the input for the
        # Reduce step, i.e. the input for reducer.py
        #
        # tab-delimited; the trivial word count is 1
        for word in words:
            print('%s%s%d' % (word, separator, 1))
if __name__ == "__main__":
    main()
```

Ukážka programu spracovávaného v MapReduce klusteri (5) – Reducer

```
#!/usr/bin/env python
"""A more advanced Reducer, using Python
    iterators and generators."""
from itertools import groupby
from operator import itemgetter
import sys
def read_mapper_output(file, separator='\t'):
    for line in file:
        yield line.rstrip().split(separator, 1)
def main(separator='\t'):
    # input comes from STDIN (standard input)
    data = read_mapper_output(sys.stdin,
     separator=separator)
    # groupby groups multiple word-count pairs
     by word,
    # and creates an iterator that returns
     consecutive keys and their group:
    #   current_word - string containing a
     word (the key)
    #   group - iterator yielding all
     ["&lt;current_word&gt;",
     "&lt;count&gt;"] items
    for current_word, group in
     groupby(data, itemgetter(0)):
        try:
            total_count = sum(int(count) for
     current_word, count in group)
            print("%s%s%d" % (current_word,
     separator, total_count))
        except ValueError:
            # count was not a number, so
     silently discard this item
            pass
if __name__ == "__main__":
    main()
```

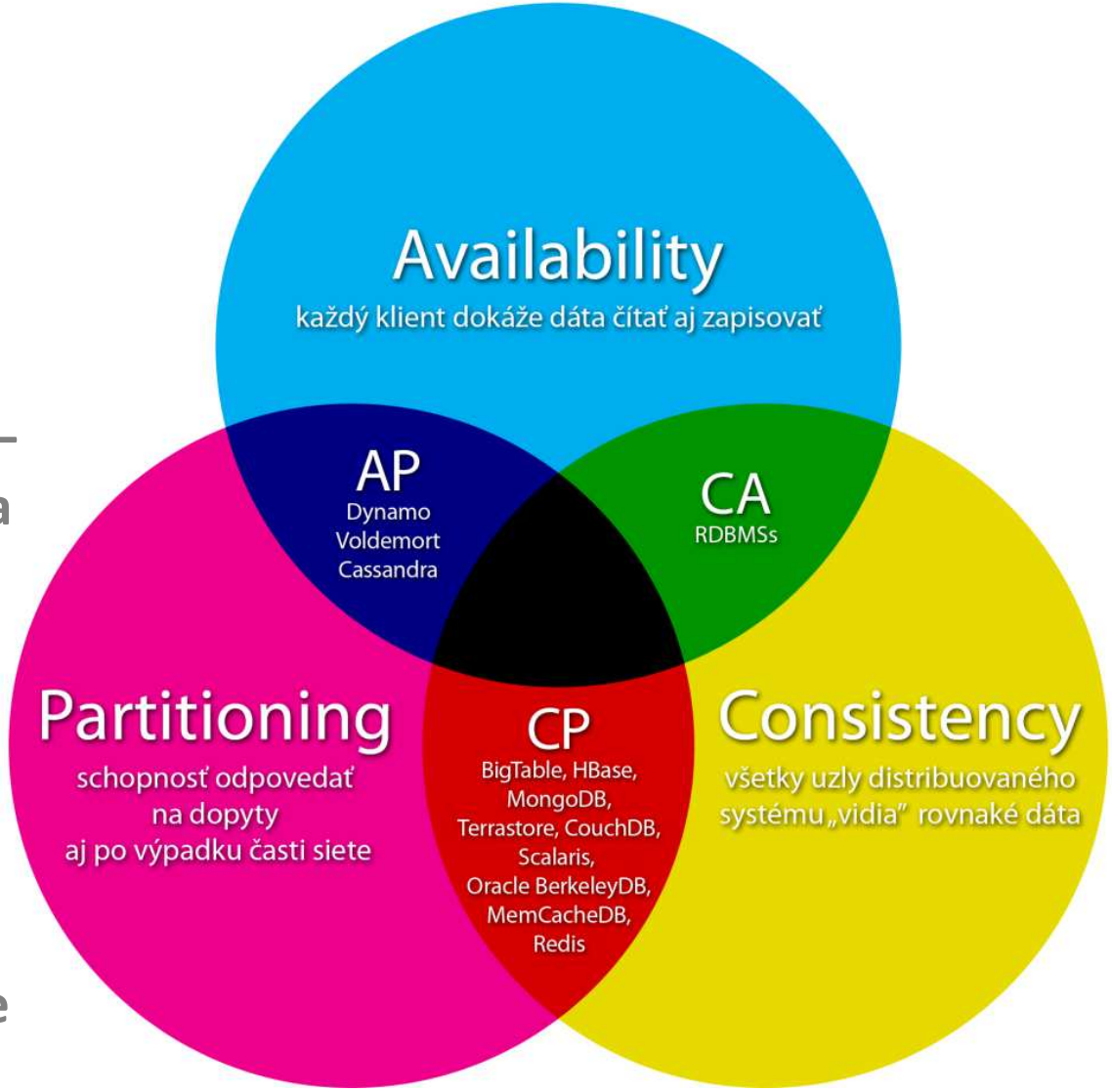
Spustenie úlohy v systéme Hadoop

- Spustenie MapReduce úlohy v Hadoop klusteri – Java
  - **hadoop jar name,jar [parameters] input output**
  - **hadoop jar /usr/local/hadoop-2.9.2/share/hadoop/mapreduce/hadoop-mapreduce-examples-2.9.2.jar wordcount hviezdoslav/gabor.txt gabor_wordcount**
  - **hadoop fs -cat gabor_wordcount/*|more**
- Spustenie MapReduce úlohy v Hadoop klusteri – Python
  - **echo "foo foo quux labs foo bar quux" | python3 mapper.py**
  - **echo "foo foo quux labs foo bar quux" | python3 mapper.py | sort -k1,1 | python3 reducer.py**
  - **hadoop jar /usr/local/hadoop-2.9.2/share/hadoop/tools/lib/hadoop-streaming-2.9.2.jar -file mapper.py -mapper mapper.py **
    **-file reducer.py -reducer reducer.py **
    -input hviezdoslav -output hviezdoslav-output

DISTRIBUOVANÉ DATABÁZY

Distribuované databázy podľa prístupu

- BigTable (Google)
  - Konzistentná
  - Odolná voči výpadkom uzlov
- Dynamo (Amazon)
  - Vždy dostupná (čítanie aj zápis – klient dostane info o tom či bola operácia úspešná alebo nie)
  - Odolná voči výpadkom uzlov
- Consistency, Availability, Partition tolerance (Brewerova CAP teoréma)
  - Distribuovaná databáza nemôže na 100% splniť všetky 3 požiadavky



Distribuované databázy podľa spôsobu ukladania dát

- **Stĺpcovo orientované distribuované databázy**
  - BigTable
  - HBase
  - HyperTable
  - Cassandra
- **Dokumentovo orientované distribuované databázy**
  - MongoDB
  - Terrastore
  - CouchDB
- **Distribuované databázy typu kľúč/hodnota**
  - Scalaris
  - Oracle Berkeley DB
  - MemcacheDB
  - Redis
  - Dynamo
  - Voldemort

Distribuované databázy podľa BigTable (CP)
Stĺpcovo orientované databázy

- HBase
- HyperTable
- Pri stĺpcovo orientovaných databázach sú tabuľky fyzicky uložené po stĺpcoch
  - **stĺpce môžu obsahovať ľubovoľný obsah**
  - **stĺpce je možné ďalej deliť**

Distribuované databázy podľa BigTable (CP)
HBase (Stĺpcovo orientovaná databáza) (1)

- Postavená nad HDFS
- Master/slave architektúra
- Master je SPOF
  - Odstránené v novších verziách (Apache Zookeeper)
  - SPOF ale naďalej zostáva master v HDFS (Apache Zookeeper)

Distribuované databázy podľa BigTable (CP) HBase (Stĺpcovo orientovaná databáza) (2)

- Ukážka tabuľky uloženej v systéme Hbase – konceptuálny pohľad

Row Key	Time Stamp	Column *"contents:"*	Column *"anchor:"*		Column *"mime:"*
"com.cnn.www"	t9		"anchor:cnnsi.com"	"CNN"	
	t8		"anchor:my.look.ca"	"CNN.com"	
	t6	"<html>..."			"text/html"
	t5	"<html>..."			
	t3	"<html>..."			

- Ukážka tabuľky uloženej v systéme HBase – fyzické uloženie v HDFS

Row Key	Time Stamp	Column *"contents:"*
"com.cnn.www"	t6	"<html>..."
	t5	"<html>..."
	t3	"<html>..."

Row Key	Time Stamp	Column *"anchor:"*	
"com.cnn.www"	t9	"anchor:cnnsi.com"	"CNN"
	t8	"anchor:my.look.ca"	"CNN.com"

Row Key	Time Stamp	Column *"mime:"*
"com.cnn.www"	t6	"text/html"

Distribuované databázy podľa BigTable (CP) HBase (Stĺpcovo orientovaná databáza) (3)

- HBase Shell
  - **SHOW tables;**
  - **DROP table;**
  - **SELECT data: FROM emails_data WHERE row='1_IR_2';**
  - **…**

Distribuované databázy podľa BigTable (CP)
Dokumentovo orientované databázy

- MongoDB
- Terrastore
- Čo je to dokument
  - Meno="Martin", Priezvisko="Seleng", Vek=35, Adresa="Kvietkova ulica 5"
  - Meno="Ferko", Priezvisko="Mrkvicka", Vek=48, Adresa="Konvalinkova ulica 2", Deti=[{Meno:"Anna", Vek:12}, {Meno:"Jozef", Vek:7}]
- Pripomína to stĺpcovú databázu (stĺpce sa môžu ľubovoľne deliť)
- Hlavné techniky na čítanie a zapisovanie do dokumentovo orientovaných databáz sú HTTP služby používajúce najmä štandardy JSON alebo XML.
- XML databázy sú dokumentovo orientované databázy

Distribuované databázy podľa BigTable (CP)
Distribuované databázy typu kľúč/hodnota (1)

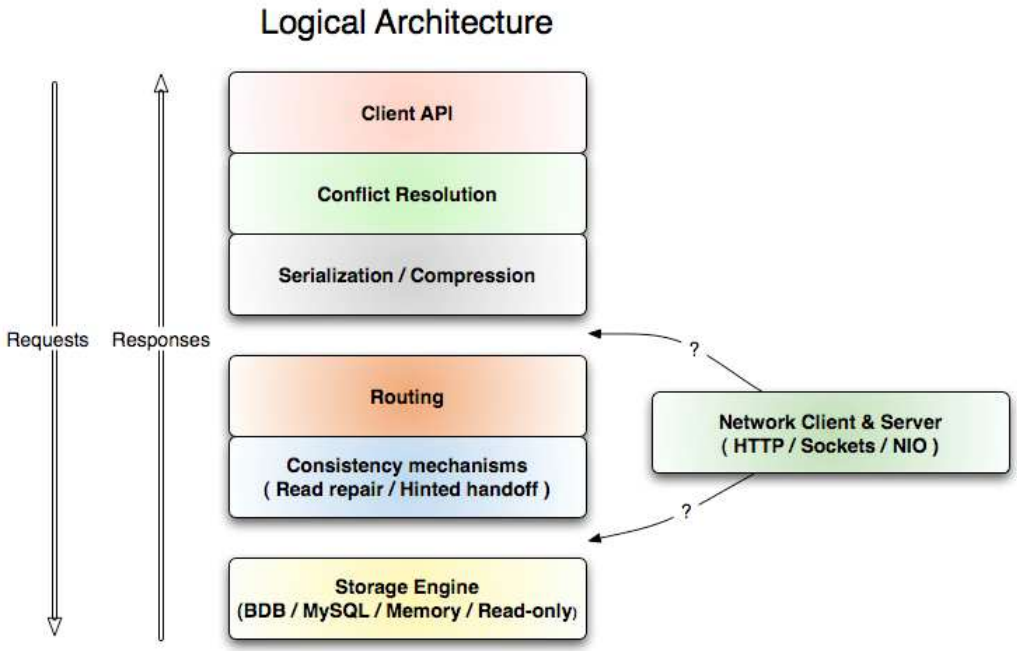
- Scalaris
- Oracle Berkeley DB
- MemcacheDB
- Redis
- Kľúč/hodnota (Key/Value) databázy, majú jednoduchú štruktúru, pretože obsahujú vždy len dve položky:
  - **kľúč, ktorý je jedinečný,**
  - **a hodnotu, ktorá je priradená k tomuto kľúču a môže byť akákoľvek.**

Distribuované databázy podľa Dynama (AP)
Distribuované databázy typu kľúč/hodnota (1)

- Spoločnosť Amazon potrebovala vyriešiť správu veľkého množstva produktov ponúkaných na svojej stránke
  - **Relačné databázy nevyhovovali z dôvodu neprispôsobenia sa distribuovanému prostrediu (pôsobnosť spoločnosti Amazon je prakticky celý svet)**
  - **Riešenie spoločnosti Amazon preferuje informovanie klienta či ním vykonávaná operácia bola úspešná alebo nie pred konzistenciou dát**
  - **Dáta zapísané do systému nie sú zablokované pokiaľ sa zapíšu všetky repliky (možnosť inkonzistencie)**
  - **Uvedenie tzv. konzistencie v čase resp., vytvorenie kvóra ktoré po zapísaní určitého počtu replík do systému povolí aj čítanie**
- Voldemort (LinkedIn)

Distribuované databázy podľa Dynama (AP) Voldemort (distribuovaná databáza kľúč/hodnota)

- Architektúra (logické bloky, open source)
- Každý z týchto blokov je zodpovedný za všetky operácie, ktoré v týchto systémoch existujú (read/get, write/put/store, delete)
- Voldemort sa snaží riešiť aj konzistenciu dát, ktorá ale nie je na 100% zaručená
- V prípade operácie write sa replikovanie vykonáva offline-ovo a vzniknuté inkonzistencie sa riešia až v prípade operácie read
- pridáva ku každej operácii write časový vektor, ktorý tvorí pár server:verzia (časová značka nemôže byť použitá, pretože v distribuovanom systéme sa uzly objavujú a miznú, replikácia trvá nejaký čas, atď.)



Distribuované databázy podľa Dynama (AP) Cassandra (stĺpcovo orientovaná databáza)

- Tvorca projektu Cassandra je spoločnosť Facebook!
- Vznikla s potrebou výkonného škálovateľného a spoľahlivého riešenia dátového úložiska vo firme Facebook na zabezpečenie niektorých služieb vyžadujúcich nízku latenciu, ako je napríklad **Inbox Search**
- Služba **Inbox Search** umožňuje vyše 500 mil. používateľom vyhľadávať v správach podľa mena odosielateľa alebo kľúčových slov
- Používa kruhový systém replikácie a skladovania dát
  - **Dopredu je určené ktorý uzol bude slúžiť na ukladanie ktorých, kľúčov**
  - **V prípade pridania nového uzla nie sú dáta automaticky prereplikované na tento uzol (priradí sa uzlu rozsah dát a nové dáta sú naňho automaticky ukladané)**
- Neskôr sa Facebook vrátil späť k CP databáze a to konkrétne Hbase (zamestnali šéfa projektu Hbase)

Distribuované databázy podľa Dynama (AP)
CouchDB (dokumentovo orientovaná databáza)

- Mnoho klientskych aplikácií v rôznych jazykoch

- Dopyty sú JSON objekty

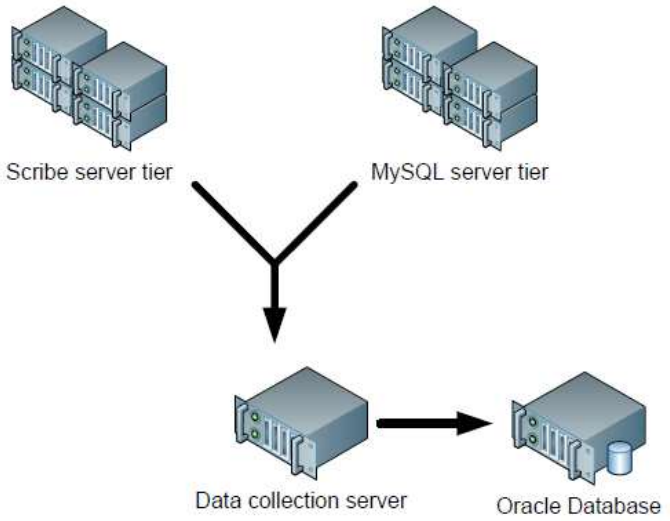
HIVE

Obsah

- Úvod – Hive vznik a funkcia
- Ukážka programu spracovávaného v MapReduce klusteri s dátovým skladiskom Hive
- Záver

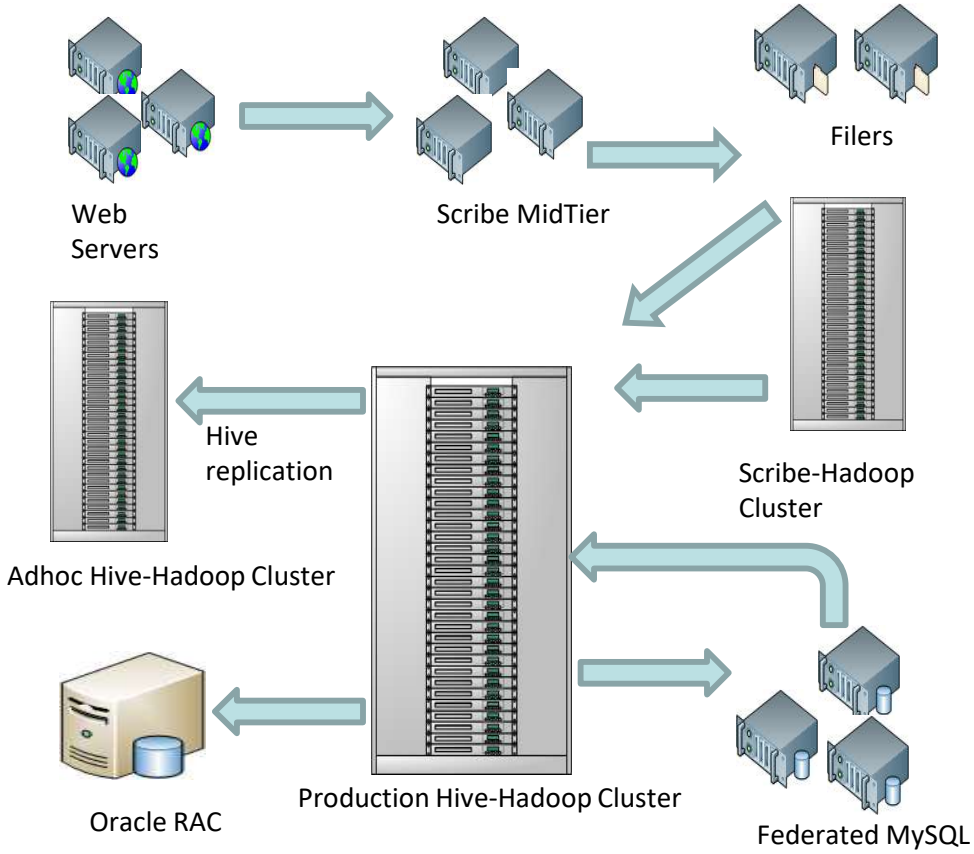
Hive – kedysi (-2012)

- Facebook
  - **Postup zbierania dát vyprodukovaný používateľmi v rámci sociálnej siete Facebook**
    - Dáta boli zbierané pomocou úloh zadaných v plánovači (boli to úlohy spúšťané v nočných časoch) a dáta boli zbierané do Oracle databázy
    - ETL (Extract, transform a load) úlohy boli naimplementované v Pythone
  - **Množstvo spracovávaných dát**
    - 2006 – niekoľko 10GB
    - 2008 – okolo 200GB nových dát
    - 2009 – okolo 5TB (skomprimovaných) nových dát denne
  - **Použitie systému Hive**
    - Spracovanie logov,
    - dolovanie textových informácií,
    - indexovanie dokumentov,
    - modelovanie správania sa používateľov (túto vec má rád môj priateľ, možno ju budeš mať aj ty) a testovanie rôznych hypotéz.



Hive – dnes (2012) – tiež kedysi

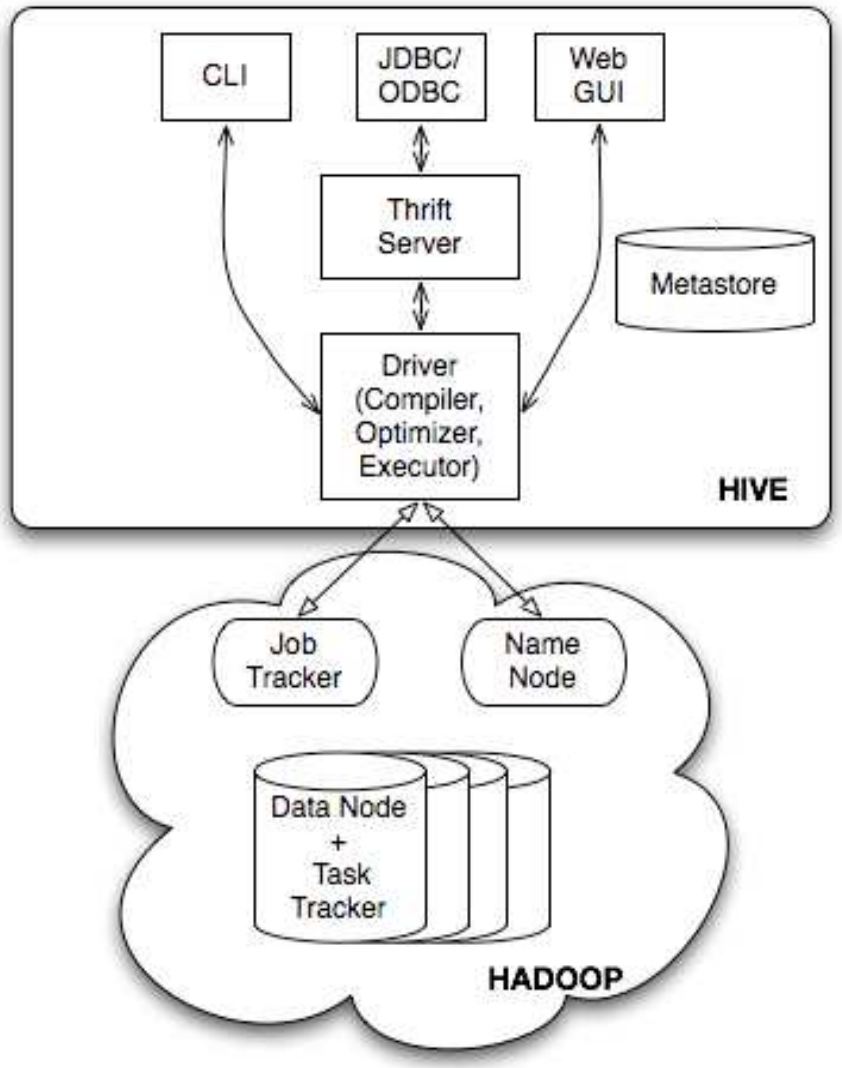
- Hadoop/Hive dátové skladisko
  - 5800 jadier, disková kapacita 8.7PB
  - 12 TB na uzol
  - Dve úrovne pripojenia topológie siete
    - 1 Gbit/s z uzla do switchu v danom racku
    - 4 Gbit/s do hlavnej úrovne zo switchu racku
- Štatistika (deň):
  - 12 TB pridaných komprimovaných údajov
  - 135TB prezeraných komprimovaných údajov
  - 7500+ spustených Hive úloh
  - 80 000 hodín strojového času
- Hive zjednodušuje Hadoop:
  - ~200 ľudí/mesiac spúšťa úlohy nad Hadoop/Hive
  - Analytici (nie informatici) používajú Hadoop pomocou Hive
  - 95% úloh je Hive úloh



Čo Hive poskytuje

- Samotný Hadoop (MapReduce) spolu s HDFS, už poskytuje možnosť spracovávať veľké množstvá dát nahrané do HDFS
- Čo však Hadoop neposkytuje je:
  - **jazyk, ktorým by sa dalo jednoducho tieto úlohy písať (bez potreby písať MapReduce programy),**
  - **editor príkazového riadku, v ktorom by sa tieto úlohy mohli písať,**
  - **schémy o jednotlivých tabuľkách v databázach**
- Na všetky tieto otázky dáva Hive odpoveď
  - **Poskytuje vlastný editor príkazového riadku (tzv. hive>), ktorý je podobný MySQL editoru (mysql>)**
  - **Jazyk, ktorým je možné písať dopyty (HQL – Hive query language, podobné SQL)**
  - **Podporu pre JDBC klientov**
  - **Uloženie metadát o databázach a tabuľkách**
  - **Možnosť písať SQL dopyty, pričom Hive automaticky preloží tieto dopyty do Map a Reduce úloh**
  - **Dáta sú štandardne csv súbory, ale je možné použiť ľubovoľné objekty**

Architektúra systému Hive



Ukážka práce so systémom Hive (1)

- http://zlatyfond.sme.sk, pričom sme použili 3 diela od Pavla Országha-Hviezdoslava: Hájnikova žena, Ežo Vlkolinský a Gábor Vlkolinský
- MapReduce klaster s implementáciou Hadoop inštalovaný na UISAV
- 14 pracovných uzlov (slaves) a 1 riadiaci server (master)
- Postup
  - **Predspracovanie dát ako napr.: konverzia pdf do textovej podoby, vyhodenie diakritiky a konverzia veľkých znakov na malé**
  - **Nahratie dát do HDFS.**
    - hadoop fs -copyFromLocal ezo.txt ezo.txt
    - hadoop fs -copyFromLocal gabor.txt gabor.txt
    - hadoop fs -copyFromLocal zena.txt zena.txt
  - **Spočítanie slov (neberieme do úvahy slovenčinu, teda žiaden stemmer ani lematizátor) v jednotlivých dielach**
    - hadoop jar /usr/local/hadoop-2.9.2/share/hadoop/mapreduce/hadoop-mapreduce-examples-2.9.2.jar grep ezo.txt ezo_freq '\w+'
    - hadoop jar /usr/local/hadoop-2.9.2/share/hadoop/mapreduce/hadoop-mapreduce-examples-2.9.2.jar grep gabor.txt gabor_freq '\w+'
    - hadoop jar /usr/local/hadoop-2.9.2/share/hadoop/mapreduce/hadoop-mapreduce-examples-2.9.2.jar grep zena.txt zena_freq '\w+'

Ukážka práce so systémom Hive (2)

- Prejdeme do editora príkazového riadku systému Hive
  - $hive
  - hive> CREATE TABLE zena (freq INT, word STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t' STORED AS TEXTFILE;
  - hive> CREATE TABLE ezo (freq INT, word STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t' STORED AS TEXTFILE;
  - CREATE TABLE gabor (freq INT, word STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t' STORED AS TEXTFILE;
  - hive> LOAD DATA INPATH "zena_freq" INTO TABLE zena;
  - hive> LOAD DATA INPATH "ezo_freq" INTO TABLE ezo;
  - hive> LOAD DATA INPATH "gabor_freq" INTO TABLE gabor;

Ukážka práce so systémom Hive (3)

- **Vypísanie obsahu tabuliek ezo**
  - hive> SELECT * FROM ezo SORT BY freq DESC LIMIT 10;

Všetky znaky malé		Aj veľké znaky	
Slovo	Počet	Slovo	Počet
sa	656	sa	656
a	598	a	440
i	427	v	362
v	375	i	352
na	322	na	303
co	271	si	242
to	254	co	232
si	245	to	214
len	201	s	182
tak	195	len	172

- **Najčastejšie sa vyskytujúce frekvencie slov v diele Hájnikova žena**
  - hive> SELECT freq, COUNT(1) AS f2 FROM zena GROUP BY freq SORT BY f2 DESC;

Všetky znaky malé		Aj veľké znaky	
Počet slov	Počet výskytov	Počet slov	Počet výskytov
8727	1	9153	1
1805	2	1854	2
691	3	704	3
326	4	355	4
190	5	191	5
137	6	137	6
83	7	94	7
75	8	66	8
38	11	38	11
38	9	37	9
38	10	36	10

Ukážka práce so systémom Hive (4)

- Orientovaný acyklický graf vykonávania „SQL“
  - EXPLAIN SELECT freq, COUNT(1) AS f2 FROM zena GROUP BY freq SORT BY f2 DESC;
    - ABSTRACT SYNTAX TREE:
    - (TOK_QUERY (TOK_FROM (TOK_TABREF zena)) (TOK_INSERT (TOK_DESTINATION (TOK_DIR TOK_TMP_FILE)) (TOK_SELECT (TOK_SELEXPR (TOK_TABLE_OR_COL freq)) (TOK_SELEXPR (TOK_FUNCTION COUNT 1) f2)) (TOK_GROUPBY (TOK_TABLE_OR_COL freq)) (TOK_SORTBY (TOK_TABSORTCOLNAMEDESC (TOK_TABLE_OR_COL f2)))))
    - STAGE DEPENDENCIES:
    - Stage-1 is a root stage
    - Stage-2 depends on stages: Stage-1
    - Stage-0 is a root stage
    - ....

Ukážka práce so systémom Hive (5)

- Ukážka na príkaz JOIN
  - hive> CREATE TABLE spojena (word STRING, ezo_f INT, zena_f INT) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
  - hive> INSERT OVERWRITE TABLE spojena SELECT e.word, e.freq, z.freq FROM ezo e JOIN zena z ON (e.word = z.word);
- Ktoré slovo sa najčastejšie vyskytuje v obidvoch dielach spolu
  - hive> SELECT word, ezo_f, zena_f, (ezo_f + zena_f) AS s FROM spojena SORT BY s DESC LIMIT 10;

Všetky znaky malé				Aj veľké znaky			
Slovo	Počet slov Ežo	Počet slov Žena	Počet slov spolu	Slovo	Počet slov Ežo	Počet slov Žena	Počet slov spolu
sa	656	970	1626	sa	656	970	1626
a	598	865	1463	a	440	742	1182
V	375	809	1184	v	362	786	1148
i	427	521	948	na	303	499	802
na	322	535	857	i	352	420	772
co	271	347	618	co	232	281	513
jak	118	432	550	jak	113	381	494
to	254	248	502	to	214	214	428
len	201	242	443	si	242	166	408
tak	195	238	434	z	158	249	407

Ukážka práce so systémom Hive (6)

- Ukážka na príkaz JOIN
  - hive> CREATE TABLE spojena1 (word STRING, ezo_f INT, zena_f INT, gabor_f INT) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
  - hive> INSERT OVERWRITE TABLE spojena1 SELECT DISTINCT e.word, e.freq, z.freq, g.freq FROM ezo e JOIN zena z ON (e.word = z.word) JOIN gabor g ON (e.word = g.word);
- Ktoré slovo sa najčastejšie vyskytuje vo všetkých troch dielach spolu
  - hive> SELECT word, ezo_f, zena_f, gabor_f, (ezo_f + zena_f + gabor_f) AS s FROM spojena1 SORT BY s DESC LIMIT 10;

Slovo	Počet slov Ežo	Počet slov Žena	Počet slov Gábor	Počet slov spolu
sa	656	970	876	2502
a	598	865	614	2077
v	375	809	800	1984
i	322	535	724	1672
na	427	521	592	1449
co	271	347	307	925
jak	118	432	261	811
z	163	262	375	800
to	254	248	254	756
len	201	242	295	738

PIG Latin – logy z Yahoo! servera

- 2A9EABFB35F5B954 970916105432 +md foods +proteins
- BED75271605EBD0C 970916025458 yahoo caht
- BED75271605EBD0C 970916090700 hawaii chat universe
- BED75271605EBD0C 970916094445 yahoo chat
- 824F413FA37520BF 970916185605 exhibitionists
- 824F413FA37520BF 970916190220 exhibitionists
- 824F413FA37520BF 970916191233 exhibitionists
- 7A8D9CFC957C7FCA 970916064707 duron paint
- 7A8D9CFC957C7FCA 970916064731 duron paint
- A25C8C765238184A 970916103534 brookings
- A25C8C765238184A 970916104751 breton liberation front
- ...

PIG Latin (1)

- Hlavnými vývojármi sú/boli zamestnanci spoločnosti Yahoo!
- Umožniť písanie map/reduce úloh (ne)odborníkom na Javu a technológiu Map/Reduce resp. expertom na relačné DB
- hadoop fs -copyFromLocal /usr/local/pig-0.17.0/tutorial/data/excite-small.log .
- $pig
- grunt> log = LOAD 'excite-small.log' AS (user, time, query);
- grunt> grpd = GROUP log BY user;
- grunt> cntd = FOREACH grpd GENERATE group, COUNT(log);
- grunt> STORE cntd INTO 'user-output';

- …
- grunt> cntd = FOREACH grpd GENERATE group, COUNT(log) AS cnt;
- grunt> fltrd = FILTER cntd BY cnt > 50;
- grunt> STORE fltrd INTO 'user-output1';

PIG Latin (2)

```
-- Query Phrase Popularity (Hadoop cluster)
REGISTER ./tutorial.jar;
raw = LOAD 'excite.log.bz2' USING PigStorage('\t') AS (user, time, query);
clean1 = FILTER raw BY org.apache.pig.tutorial.NonURLDetector(query);
clean2 = FOREACH clean1 GENERATE user, time, org.apache.pig.tutorial.ToLower(query) as query;
houred = FOREACH clean2 GENERATE user, org.apache.pig.tutorial.ExtractHour(time) as hour, query;
ngramed1 = FOREACH houred GENERATE user, hour, flatten(org.apache.pig.tutorial.NGramGenerator(query)) as
ngram;
ngramed2 = DISTINCT ngramed1;
hour_frequency1 = GROUP ngramed2 BY (ngram, hour);
hour_frequency2 = FOREACH hour_frequency1 GENERATE flatten($0), COUNT($1) as count;
uniq_frequency1 = GROUP hour_frequency2 BY group::ngram;
uniq_frequency2 = FOREACH uniq_frequency1 GENERATE flatten($0),
flatten(org.apache.pig.tutorial.ScoreGenerator($1));
uniq_frequency3 = FOREACH uniq_frequency2 GENERATE $1 as hour, $0 as ngram, $2 as score, $3 as count, $4 as
mean;
filtered_uniq_frequency = FILTER uniq_frequency3 BY score > 2.0;
ordered_uniq_frequency = ORDER filtered_uniq_frequency BY hour, score;
-- Output: (hour, n-gram, score, count, average_counts_among_all_hours)
STORE ordered_uniq_frequency INTO 'script1-hadoop-results' USING PigStorage();
```

SPARK

Apache Spark história

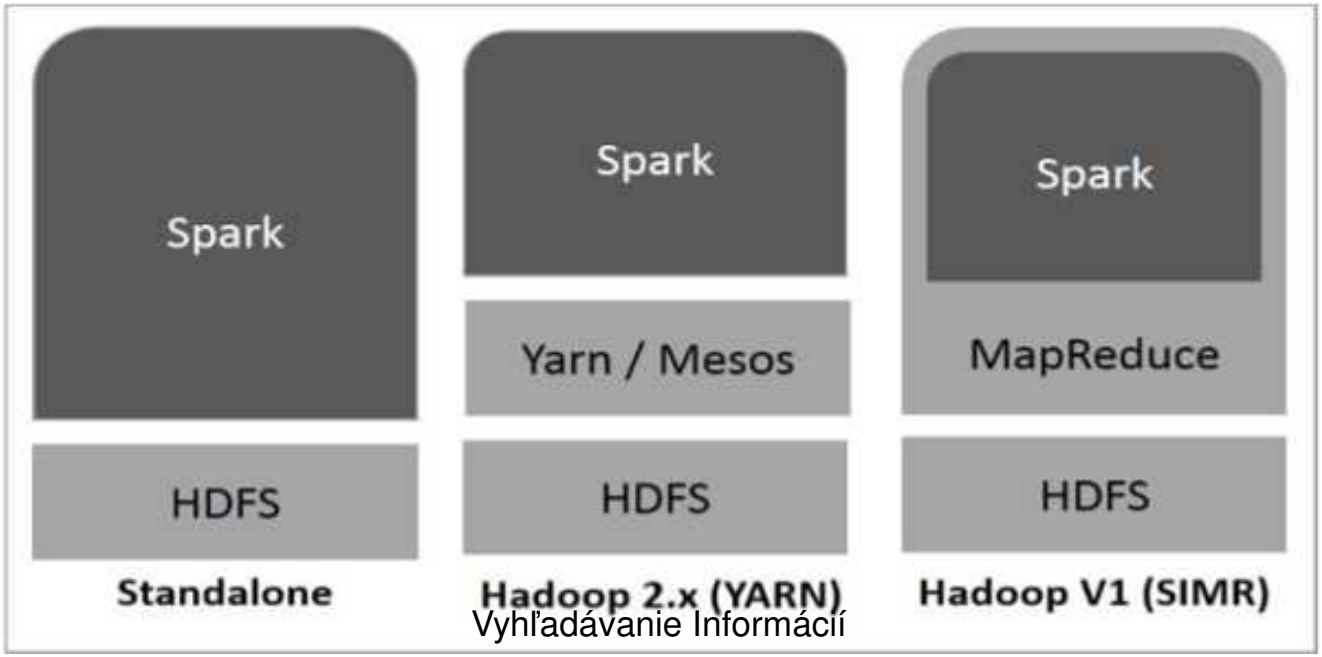
- Spark je jeden z podprojektov Hadoopu vyvinutý v 2009 na univerzite v Berkeley v AMPLabe (Matei Zaharia)
- Stal sa open sourcom v 2010 BSD licencia
- V 2013 bol venovaný nadácii Apache
- Apache Spark sa stal top projektom vo februári 2014

Apache Spark funkcionalita

- **Apache Spark™**
  - zjednotené riešenie pre spracovanie veľkých dát
  - in-memory cluster computing
- **Features**
  - Rýchlosť – Spark umožňuje bežať aplikácie nad Hadoop klastrom až 100x rýchlejšie (ak beží v pamäti), a 10x rýchlejšie (na disku). Toto je dosiahnuté znížením IO diskových operácií, priebežné výsledky si medzi-výsledky do pamäte.
  - Podporuje rôzne programovacie jazyky – Spark poskytuje API v Jave, Scale, alebo Pythone. Spark obsahuje do 80 high-level operátorov pre interaktívne dopytovanie.
  - Rozšírená Analytika – Spark neposkytuje iba metódy „Map“ A „Reduce“, ale podporuje aj SQL dopyty, spracovanie prúdov dát, strojové učenie a grafové algoritmy.

Apache Spark – spôsoby spustenia

- **Standalone** – Pri Spark Standalone inštalácii je Spark spustený priamo nad HDFS. Ak by bol na klastri naištalovaný aj Hadoop, tak Spark a Hadoop bežia vedľa seba paralelne.
  - **Nevýhoda, jeden klaster nevie koľko zdrojov obsadil druhý.**
- **Hadoop Yarn** – Hadoop Yarn inštalácia znamená, že Spark je je spustený priamo na Hadoop Yarn klastri, bez potreby nejakého inštalovania, alebo root prístupu.
  - **Umožňuje integrovať Spark priamo do Hadoop ekosystému.**
  - **Zároveň umožňuje aj iným komponentom bežať nad týmto stackom.**
- **Spark in MapReduce (SIMR)** – SIMR poskytuje jednoduchý a rýchly spôsob pre používateľov Hadoopu MRv1 deploynuť a použiť Apache Spark.
  - **MRv1 sa dnes už veľmi nepoužíva**



Apache Spark komponenty

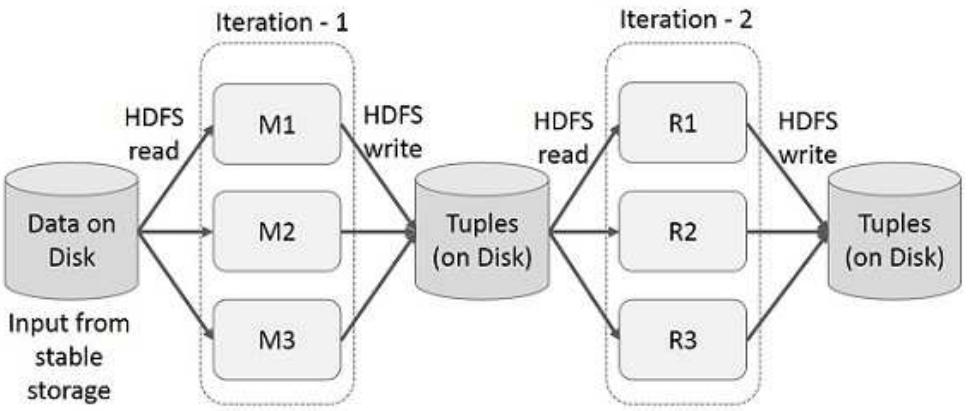
- **Apache Spark Core**
  - Spark Core je základ Apache Sparku, na ktorom sú ostatné komponenty postavené. Poskytuje In-Memory spracovanie a IO operácie do externých ukladacích systémov.
- **Spark SQL**
  - Spark SQL je komponent, ktorý definuje dátovú abstrakciu nazvanú SchemaRDD, ktorá poskytuje podporu pre spracovanie štruktúrovaných a semi-štruktúrovaných dát.
- **Spark Streaming**
  - Spark Streaming využíva schopnosť Spark Core rýchlo pripraviť a vykonať analýztu prúdov údajov. Načítava dáta v mini-batches a vykonáva RDD (Resilient Distributed Datasets) transformácie nad nimi
- **MLlib (Machine Learning Library)**
  - MLlib je distribuovaný machine learning framework nad Sparkom. Spark MLlib je 9x rýchlejší ako Apache Mahout (predtým ako Mahout dostal interfejs do Sparku), ktorý pracuje priamo nad diskom.
- **GraphX**
  - GraphX je framework umožňujúci distribuované spracovanie grafov nad Sparkom. Poskytuje API na spracovanie grafov používateľov pomocou abstrakcie Pregel API.

RDD

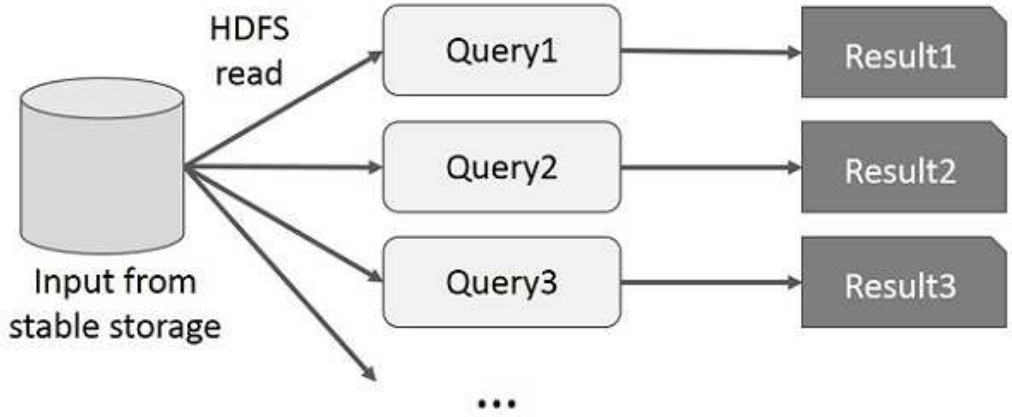
- Resilient Distributed Datasets (RDD) je základnym dátovým objektom v Sparku. Je to nemenná distribuovaná kolekcia objektov. Každý RDD je rozdelený na logické partície, ktoré môžu byť spracovávané na rôznych uzloch klastra. RDD môžu obsahovať rôzne typy objektov z Pythonu, Javy, alebo Scaly, ako aj používateľom definované objekty.
- Formálne RDD je iba na čítanie, rozdelená kolekcia záznamov. RDD môžu byť vytvorené použitím deterministických operácií z dát uložených na diskoch, alebo z iných RDD. RDD je kolekcia fault-tolerant objektov, ktoré môžu byť spracovávané paralelne.
- Sú dve možnosti ako vytvoriť RDD
  - **Paralelizovaním existujúcej kolekccie v driver časti Sparku**
  - **Referencovaním datasetu v externom úložisku, ako sú: HDFS, HBase, alebo akékoľvek zdroje dát, poskytujúce HadoopInputFormat.**
- Spark používa koncept RDD k vytvoreniu rýchlejších a efektívnejších MapReduce operácií.

Výmena dát je v MapReduce pomalá

- Bohužiaľ vo väčšine súčasných frameworkov, jediný spôsob ako zdieľať dáta medzi rôznymi výpočtami (medzi dvomi MapReduce úlohami) je zapísať ich na disk (HDFS).
- Iteratívne ako aj Interaktívne aplikácie vyžadujú rýchlu výmenu dát medzi úlohami.
  - **Výmena dát je pomalá v MapReduce kvôli replikácii a diskovým IO operáciam.**
  - **Väčšina Hadoop aplikácií strávi okolo 90% času HDFS r/w operáciami.**
- Iteratívne operácie

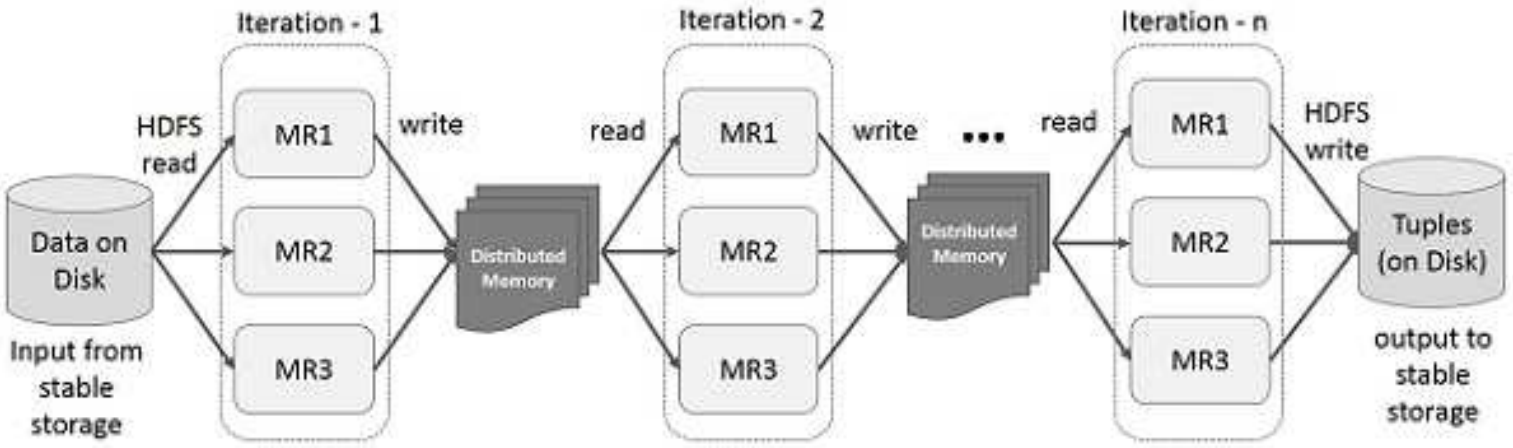


Interaktívne operácie

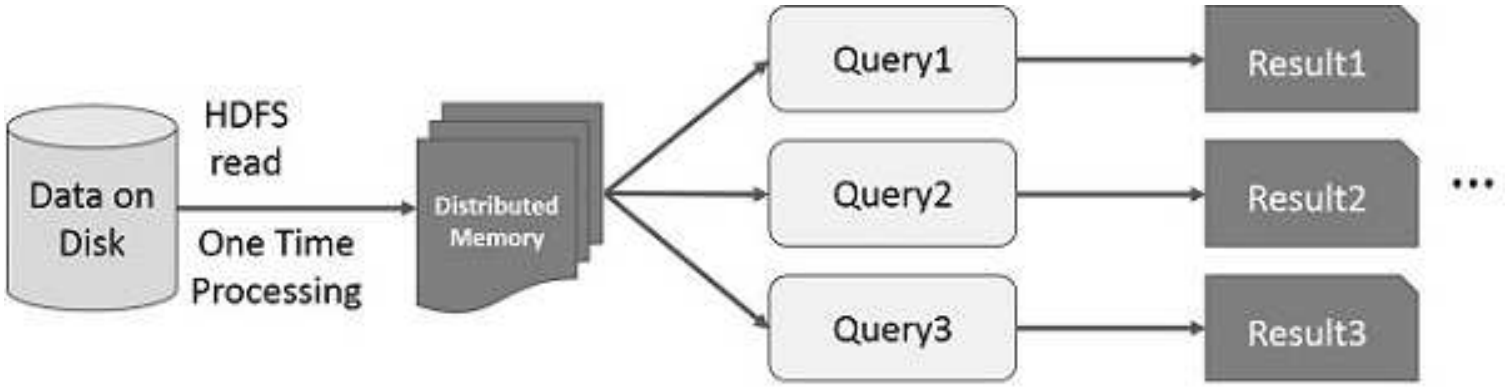


Výmena dát použitím Spark RDD

- Iteratívne operácie



- Interaktívne operácie



Transformácie a akcie v Sparku (1)

- map(func)
- filter(func)
- flatMap(func)
- union(otherDataset)
- intersection(otherDataset)
- distinct([numTasks])
- groupByKey([numTasks])
- reduceByKey(func, [numTasks])
- aggregateByKey(zeroValue)(seqOp, combOp, [numTasks])
- sortByKey([ascending], [numTasks])
- join(otherDataset, [numTasks])
- cogroup(otherDataset, [numTasks])
- cartesian(otherDataset)
- pipe(command, [envVars])
- coalesce(numPartitions)
- repartition(numPartitions)
- repartitionAndSortWithinPartitions(partitioner)

Transformácie a akcie v Sparku (2)

- reduce(func)
- collect()
- count()
- first()
- take(n)
- takeSample (withReplacement,num, [seed])
- takeOrdered(n, [ordering])
- saveAsTextFile(path)
- saveAsSequenceFile(path) (Java and Scala)
- saveAsObjectFile(path) (Java and Scala)
- countByKey()
- foreach(func)

Ukážka kódu v Sparku (1)

```
spark-shell
val NUM_SAMPLES=100000000
val count = sc.parallelize(1 to NUM_SAMPLES).filter { _ =>
  val x = math.random
  val y = math.random
  x*x + y*y < 1
}.count()
println(s"Pi is roughly ${4.0 * count / NUM_SAMPLES}")
```

```
pyspark --master spark://127.0.0.1:6066 --executor-memory 512m
import random
NUM_SAMPLES = 100000
def inside(p):
    x, y = random.random(), random.random()
    return x*x + y*y < 1

count = sc.parallelize(range(0, NUM_SAMPLES)).filter(inside).count()
print("Pi is roughly %f" % (4.0 * count / NUM_SAMPLES))
```

Ukážka kódu v Sparku (2)

```
spark-shell
val inputfile = sc.textFile("ezo.txt")
val counts = inputfile.flatMap(line => line.split("
")).map(word => (word, 1)).reduceByKey(_ + _)
counts.toDebugString
counts.cache()
counts.saveAsTextFile("output")
-------------------------------------------------
pyspark --master spark://127.0.0.1:6066 --executor-memory
512m
text_file = sc.textFile("ezo.txt")
counts = text_file.flatMap(lambda line: line.split(" ")) \
             .map(lambda word: (word, 1)) \
             .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("output1")
```